



CURSOR BUILDATHON · PROJECT SUBMISSION DOCUMENT

Cursor × TechTalk360 · Best use of n8n

Repository: github.com/Kiruthiyan/n8n_aivura · Bot: @Aivura_bot

Confidential — For Authorized Use Only

Section 01 — Project Overview

Project Name & One-Line Pitch

Aivura — An AI academic co-pilot on Telegram that turns lecture PDFs, deadlines, and study questions into structured, actionable replies for university students, orchestrated entirely in **n8n** and built with **Cursor**.

Summary

University students juggle assignments, exams, and scattered tools (email, Drive, notes apps) while deadlines slip and study time is wasted re-reading the same material. Aivura gives them one interface they already use daily: **Telegram**. Students message the bot, upload PDFs, or run slash commands; **n8n** authenticates them, classifies intent, routes to the right workflow, pulls context from Notion and Gmail where needed, and returns a consistent four-section AI response. Admins onboard students via a **Cursor-built Admin Portal** and sync access to a Notion Users database—no student signup, no extra app. The product is production-shaped: modular workflows, env-based secrets, and a workflow generator script for maintainability.

Submission Details

Item	Detail
Track	Best use of n8n by n8n
Track integration	12 n8n workflows (1 router + 11 feature sub-workflows), Telegram Trigger, Execute Workflow, Switch routing, HTTP nodes to Gemini / Notion / Gmail / Telegram File API, Code nodes for auth, PDF vision, and demo mode
Cursor integration	Full repo scaffolded and iterated in Cursor: workflow JSON generator (<code>scripts/build-aivura-student-bot.mjs</code>), React Admin Portal, docs, and security patterns
Team name	*[Fill in]*
Demo URL	https://t.me/Aivura_bot
Repository	https://github.com/Kiruthiyan/n8n_aivura

Section 02 — Problem Statement

The Problem

Who: Undergraduate and postgraduate students at Sri Lankan and regional universities (initial beachhead: tech/business faculties with heavy assignment load).

Context: During semester peaks, students receive assignment briefs via email, store files in Drive, track tasks in ad-hoc Notion boards or WhatsApp groups, and ask friends or ChatGPT in separate tabs—none connected to *their* deadlines or *their* documents.

Consequence if unsolved: Missed deadlines, shallow revision (re-reading PDFs without extraction), inconsistent assignment structure, and cognitive overload from tool-switching. Stress spikes in the two weeks before exams.

Why It Matters

Dimension	Detail
Frequency	Daily during term; multiple academic events per week per student
Severity	Direct impact on grades, mental health, and retention
Current workaround	Generic ChatGPT (no personal deadlines), manual Notion boards (not mobile-first), study groups on WhatsApp (unstructured, no document pipeline)
Why workarounds fail	No single authenticated channel tied to the student’s real academic data; no orchestration across Telegram + email + tasks

Root Cause

The **fragmentation of academic workflows across channels**—not lack of AI models. Students need orchestration and access control on top of AI, delivered where they already communicate (mobile messaging), without building another SaaS login.

Section 03 — Proposed Solution

What the Product Does

Aivura is a **Telegram-native student assistant** ([@Aivura_bot](#)). A student messages the bot or sends a PDF with a caption. **n8n** receives the webhook, verifies the Telegram Chat ID (Notion or demo allowlist), detects intent (slash command + Gemini JSON classifier), and **executes exactly one of eleven specialized sub-workflows**. Each sub-workflow gathers relevant context (document text via Gemini Vision, Gmail threads, Notion tasks), calls **Gemini 2.0 Flash** with a feature-specific system prompt, formats a structured Markdown reply, and returns it to Telegram.

Core mechanism: *Workflow-orchestrated AI*—business logic, auth, routing, and integrations live in n8n; the LLM is a step inside deterministic pipelines, not a black-box chat UI.

Key Features (built & demonstrable)

Feature	User need addressed
PDF / document summary	Turn long readings into exam-ready points without re-reading
MCQ generation	Self-test before quizzes from any topic or notes
Submission answer draft	Structured assignment outline (study aid; student verifies)
Study plan	Time-boxed plan using real tasks/deadlines when integrated
Deadlines digest	Surface urgent academic items from email + Notion
Viva prep	Likely questions + model answers before oral exams
Lecture summary	Compress lecture notes/PDFs for revision
Flashcards	Revision cards from any topic
Today plan	Next few hours study blocks
Explain simple	Hard topic in plain language (<code>/simple</code>)
General ask / help	On-demand tutoring + command discovery
Admin Portal	Add students, export to Notion, broadcast messages
Access gate	Unregistered users denied; demo mode without Notion
Gemini Vision PDF	Real PDF/image text extraction (not placeholder)

Scope

In scope (this build):

- Telegram bot (message + document)
- 12 n8n workflow files (router + 11 student sub-workflows)
- Notion-based user registry (+ demo mode bypass)
- Google Gemini 2.0 Flash for intent + responses + PDF vision
- Optional Gmail + Notion student DB for context-rich flows
- Cursor-built Admin Portal (local/Vercel-ready)
- Workflow generator for reproducible n8n JSON

Deliberately out of scope (time):

- Sinhala/Tamil full localization
- Student self-registration and payments
- Multi-university tenancy and billing
- Native mobile app (Telegram is the client)

Section 04 — Functional Requirements

Must Have (demo-critical)

ID	Requirement
M1	Telegram Trigger receives text and documents
M2	System rejects unregistered/inactive Chat IDs with clear message
M3	Registered user receives <code>/help</code> command list
M4	PDF upload triggers document-summary workflow
M5	Slash commands map to correct intent (<code>/mcq</code> , <code>/studyplan</code> , etc.)
M6	Gemini returns four-section Markdown (Summary, Findings, Actions, Draft/Plan)
M7	Router Switch executes one sub-workflow per request
M8	Admin can add student + Chat ID in portal and export for Notion
M9	All secrets via <code>\$env.*</code> in n8n (no keys in JSON)

Should Have

ID	Requirement
S1	Natural-language intent detection when no slash command
S2	Gmail fetch for deadline/study-plan workflows
S3	Notion student DB query for tasks/deadlines
S4	Telegram file download + Gemini Vision for PDF/images
S6	Demo mode (<code>AIVURA_DEMO_MODE</code>) for hackathon without Notion
S7	Admin broadcast via portal or CLI script
S5	<code>continueOnFail</code> + aggregated <code>api_errors</code> in context

Could Have / Won't Have (this build)

ID	Item	Decision
C1	Google Drive file listing	Won't — Gmail + Notion sufficient for demo
C2	Voice messages	Won't
C3	Multi-language Sinhala/Tamil	Could — post-hackathon
C4	Usage analytics dashboard	Could — portal Activity stub only
C5	Automated Notion sync from portal	Won't — manual export for hackathon reliability

Section 05 — Non-Functional Requirements

Performance

Interaction	Target
Text command (no Gmail)	5–15 s end-to-end (Gemini-bound)
Study plan / deadlines (Gmail + Notion)	15–30 s
PDF summary	10–25 s (download + AI)
Concurrent users	n8n queue handles modest campus pilot; scale via n8n workers + Gemini rate limits

Reliability & Error Handling

- HTTP nodes use `continueOnFail` + `alwaysOutputData`
- Code nodes collect `api_errors` and pass to Gemini for graceful degradation
- Denied users get fixed message (no stack traces)
- Sub-workflow failures still route to Collect Reply with fallback text

Usability

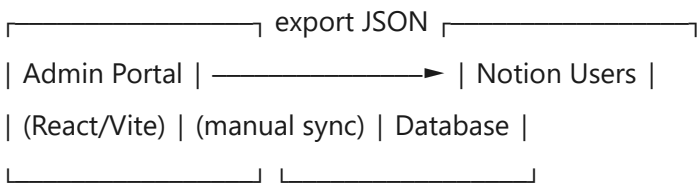
- **Students:** Zero onboarding UI—only Telegram; commands discoverable via `/help`
- **Admins:** Single portal for user CRUD and workflow documentation
- **Responses:** Predictable 4-section format reduces scan time on mobile
- **Accessibility:** Telegram client accessibility; portal uses semantic HTML and contrast-friendly dark theme

Scalability

Layer	Scale path
n8n	Horizontal workers, separate prod/staging instances
Workflows	Feature isolation—scale hot paths (PDF) independently
Notion	Per-cohort databases; pagination on queries
Gemini	Tier upgrade, caching for repeated summaries

Section 06 — Technical Architecture

System Overview



Technology Stack

Layer	Technology	Rationale
Orchestration (track)	n8n	Visual + exportable workflows; native Telegram, HTTP, Code, Execute Workflow
AI	Google Gemini 2.0 Flash	Fast, cost-effective; JSON mode for intent; vision for PDFs
Messaging	Telegram Bot API	Universal on student phones; file upload support
User registry	Notion API	Low-friction admin DB; filter by Chat ID
Tasks / context	Notion Student DB	Structured deadlines for study flows
Email context	Gmail API	Academic keyword search
Admin UI	React 18, Vite, TypeScript, Tailwind	Built rapidly in Cursor; deployable to Vercel
Workflow codegen	Node.js (<code>build-aivura-student-bot.mjs</code>)	DRY maintenance of 8 similar workflows
IDE / build	Cursor	Primary development environment for all artifacts
Secrets	n8n environment variables	No keys in repo or workflow exports

Data Flow (core action: student sends PDF + “Summarize chapter 2”)

1. **Telegram** → POST to n8n Telegram Trigger
2. **02 – Extract** → `chat_id`, `user_message`, `file_id`, `mime_type`
3. **03 – Check Registered** → Notion query `Telegram Chat ID == chat_id`
4. **04 – Lookup** → if `Active` $\neq$ true → **05a – Send Denied** → END
5. **06 – Detect Intent** → Gemini JSON `user_intent`
6. **07 – Parse Intent** → command override; PDF → `DOCUMENT_SUMMARY`
7. **08 – Switch** → **RUN** – `/summary` → Execute Workflow `01-pdf-summary`
8. Sub-workflow: download file via Telegram API → Gemini Vision (if PDF) → build context → Gemini → format
9. **09 – Collect Reply** → **10 – Telegram Reply** (Markdown)
10. Student sees four-section summary on phone

AI Integration

Use	Model	Prompting
Intent classification	Gemini 2.0 Flash, <code>responseMimeType: application/json</code>	System: enum of 11 intents; user: message + hasPdf flag
Feature responses	Gemini 2.0 Flash, temperature 0.4	Per-workflow system prompt + JSON user payload (message, context, errors)
PDF extraction	Gemini Vision inline_data	Extract text from PDF/images before summary workflow
Output contract	—	Fixed Markdown sections for mobile readability

Known Technical Limitations

- PDF body parsing uses placeholder for binary PDFs (production: Vision/OCR node)
- Portal → Notion sync is manual export (reliability for demo)
- Admin auth is password in env (hackathon); production needs SSO
- No persistent chat memory across sessions (each message is stateless)

Section 07 — Security

Authentication & Authorisation

Actor	Mechanism
Student	Telegram Chat ID matched in Notion; <code>Active</code> checkbox required
Admin	Portal password (<code>VITE_ADMIN_PASSWORD</code>); session in <code>localStorage</code>
n8n	Instance login required for operators (documented in <code>docs/security.md</code>)

No public student login surface—reduces attack area.

Data Handling

- Messages flow through n8n execution logs—operators must secure instance
- Notion stores registration metadata (name, Chat ID, role)
- Gmail: metadata + snippets only (limited fetch count)
- No passwords stored for students

API & Secret Management

- `.env.example` only placeholders; `.gitignore` blocks `.env`
- Workflow JSON uses `$env.GEMINI_API_KEY`, `$env.TELEGRAM_BOT_TOKEN`, etc.
- No secrets in Admin Portal frontend bundle except admin password variable

Input Validation

- Chat ID type coercion in Notion filter
- Command parsing via whitelist map
- Reply truncated at ~3900 chars for Telegram limits
- `continueOnFail` prevents single API failure from crashing pipeline

Known Vulnerabilities or Gaps

Gap	Mitigation plan
Admin password in client env	Move to serverless auth post-hackathon
No rate limiting	Add n8n throttle + Gemini API quotas
Telegram bot open to anyone	Notion gate; optional allowlist per cohort
Execution logs may contain PII	Retention policy + hosted n8n access control

Section 08 — User Stories & Use Cases

Core User Stories

1. **As a student**, I want to send my lecture PDF to Telegram, **so that** I get a structured summary before class.
2. **As a student**, I want `/mcq` on a topic, **so that** I can self-test without writing questions manually.
3. **As a student**, I want `/deadlines`, **so that** I see urgent items from email and my task board in one message.
4. **As a student**, I want `/studyplan` before finals, **so that** I get a realistic daily schedule.
5. **As a student**, I want `/viva`, **so that** I practice likely oral exam questions.
6. **As an admin**, I want to register a student by Chat ID, **so that** only my cohort can use the bot.
7. **As an admin**, I want to deactivate a user, **so that** access is revoked without deleting history.
8. **As a student**, I want `/help`, **so that** I know all commands without reading documentation.
9. **As a judge**, I want to see n8n route to different workflows, **so that** orchestration depth is visible in the execution graph.

Primary Use Case Walkthrough

Step	User	System
1	Admin adds "Kiru" + Chat ID in portal, exports to Notion, sets Active	Notion row created
2	Kiru opens Telegram, sends <code>/help</code>	Router → general workflow → command list
3	Student uploads <code>OS_Chapter3.pdf</code> + caption "Summarize for exam"	Router → PDF workflow → Gemini Vision → summary
4	—	Four-section Markdown reply in <30s
5	Kiru sends <code>/studyplan exams in 5 days</code>	Study plan workflow → Gmail + Notion → personalized plan

Edge Cases

Case	Behavior
Unregistered Chat ID	Polite deny + hint to contact admin
Inactive user	Same deny path
Message with no text, only PDF	Intent defaults to DOCUMENT_SUMMARY
Gmail token missing	Workflow continues; AI notes missing email context
Gemini timeout / quota	Fallback four-section error template
Unknown slash command	AI intent + fallback to GENERAL_STUDY

Section 09 — Target Users & Market

Primary User

University students (18–26) in Sri Lanka and similar markets—high smartphone penetration, heavy Telegram/WhatsApp use, English-medium technical programs. Pain: deadline anxiety and unstructured study habits.

Market Opportunity

- Sri Lanka: 50k+ university students in STEM/business alone; expandable to South/Southeast Asia
- wedge: **department-level pilot** (one faculty, one bot, admin-managed roster)
- TAM grows with institutional licenses vs. consumer subscriptions

Competitive Landscape

Alternative	Weakness vs. Aivura
Raw ChatGPT	No personal deadlines, no PDF pipeline on mobile, no access control
Notion AI	Not Telegram-native; students don't live in Notion on phone
Generic study apps	No integration with student's email/tasks; extra app install

Aivura wedge: *Orchestrated, authenticated, mobile-first academic agent*—n8n as the integration layer competitors skip in hackathon demos.

Section 10 — Business Model

Revenue Model

B2B2C via universities / private institutes — annual license per cohort (200–500 students), includes bot + admin seats + support. Fits low student willingness to pay directly.

Pricing Hypothesis

Tier	Price (hypothesis)	Includes
Pilot	Free / LKR 0 (3 months)	1 department, 100 students
Department	~LKR 150k–300k / year	500 students, Gmail optional
Campus	Custom	SSO, analytics, Sinhala/Tamil

Go-to-Market

1. **First 10 customers:** Direct outreach to 10 department heads / student union tech reps at SLIIT, Moratuwa, Colombo, NSBM
2. **Channel:** Demo during Buildathon + LinkedIn build-in-public + faculty WhatsApp groups
3. **Message:** “Your students already use Telegram—give them an AI assistant you control.”

Roadmap

Horizon	Deliverable
Now (hackathon)	12 workflows, portal, broadcast, demo mode, 11+ commands, Gemini Vision
0–3 months	PDF OCR, auto Notion sync, usage dashboard, Sinhala
12 months	Multi-tenant admin, LMS integration (Moodle), institutional billing, compliance pack

Section 11 — Why This Project Leads the Track

Technical Edge (n8n)

- **Not a single chatbot workflow** — production-style **router + 11 Execute Workflow sub-flows**
- **Real integrations:** Telegram Trigger, Notion auth (or demo mode), Gmail API, Telegram File API, Gemini ×3 (vision + intent + answer)
- **Switch node routing** on classified intent — visible, debuggable execution graph for judges
- **Workflow-as-code:** Cursor-generated `build-aivura-student-bot.mjs` regenerates all JSON— shows maintainability beyond drag-and-drop
- **Failure-aware orchestration:** parallel fetches, error aggregation, graceful AI fallbacks

Problem-Solution Fit

Students don’t need another app—they need **connected academic automation on Telegram**. n8n is the only layer that unifies messaging, auth, APIs, and AI without a custom backend team.

Execution Quality

- End-to-end demo path documented (`docs/demo-script.md`)
- Separate test matrix (`workflows/student/TELEGRAM-TESTS.md`)
- Security and architecture docs included
- Admin + student journeys both functional

Real-World Potential

Pilot-ready for a single department next week. Clear monetization, expansion roadmap, and technical path from hackathon JSON to hosted n8n Cloud + Vercel portal.

Section 12 — Team & Roles

Team Members

Name	Role	Built
[Name 1]	Lead / n8n architect	Router + sub-workflows, Notion/Gmail integration, generator script
[Name 2]	Frontend / Cursor	Admin Portal (React), UX, deployment
[Name 3]	AI / product	Prompts, intent design, demo script, submission doc

Adjust to your actual team.

Why This Team

Combination of **workflow automation** (n8n track requirement), **rapid UI** (Cursor + React), and **domain empathy** (university deadline pressure)—able to ship a full stack (bot + admin + 9 workflows) in hackathon time without a traditional backend.

Appendix — Demo Checklist (2 minutes)

1. **n8n canvas** — show Router with 8 Execute Workflow branches (10 seconds)
2. **Portal** — add student + Chat ID (15 seconds)
3. **Telegram** — unregistered → denied (10 seconds)
4. **Telegram** — `/help` (10 seconds)

5. **Telegram** — PDF summary (30 seconds)
 6. **Telegram** — `/mcq binary trees` (20 seconds)
 7. **n8n execution** — open successful run, show nodes lit up (25 seconds)
-

Cursor Buildathon · Cursor × TechTalk360 · Confidential